

PORTFOLIO

아키텍처 개선으로 트래픽 병목을 해결하는 백엔드 개발자 최준입니다.



Contact

전화번호 : 010-3815-6968

Email : choijun0627@naver.com

Blog : <https://dev-gongmyoung.tistory.com/>

Github : <https://github.com/CJ-1998>

Graduation

- [대학] 2017.03 ~ 2024.02
건국대학교 컴퓨터공학부 졸업 (3.77/4.5)

Education

- [교육] 2025.07 ~ 2026.03
삼성청년 SW AI 아카데미 14기 재학 중
- [교육] 2024.07 ~ 2024.10
내일배움캠프 Spring 심화 1기 수료
- [교육] 2023.12 ~ 2024.07
구름톤 트레이닝 풀스택 6회차 수료

Language

- [어학] 2026.03.05
OPIc (영어) IM1

Certificate

- [자격증] 2025.06.13
정보처리기사
- [자격증] 2025.04.04
SQLD
- [자격증] 2019.11.07
정보처리기능사

Skills

Language & Framework

- **Java** ★★★★★☆
객체지향 설계 기반 서비스 로직 구현
- **Spring Boot** ★★★★★☆
RESTful API 서버 구축
- **JPA** ★★★★★☆
벌크 연산 기반 DB I/O 개선

Tool

- **Git** ★★★★★☆
브랜치 전략을 활용한 소스 코드 버전 관리
- **Monitoring (PLG)** ★★★★★☆
서버 메트릭 모니터링 대시보드 구축
- **Jira** ★★★★★☆
요구사항을 세분화한 티켓 단위 작업 할당

Database

- **MySQL** ★★★★★☆
관계형 데이터 모델링 및 설계
- **Redis** ★★★★★☆
분산 락을 활용한 동시성 제어

Infra & DevOps

- **AWS** ★★★★★☆
이벤트 기반 비동기 아키텍처 구축
- **Docker** ★★★★★☆
컨테이너 기반 애플리케이션 배포
- **Jenkins** ★★★★★☆
자동화된 CI/CD 파이프라인 구축
- **Nginx** ★★★★★☆
리버스 프록시 서버 구축

PROJECTS

4# STORAGE SERVICE

백엔드 팀의 이미지 관련 처리 돕는 이미지 처리 모듈

ASKPLACE

실시간 여행지 영상 확인 가능한 여행 계획 플랫폼

4# Storage Service

백엔드 팀의 이미지 관련 처리 돕는 이미지 처리 모듈

기간 : 24.09 ~ 24.10 (5주)

팀 구성 : Backend 4명

Github : <https://github.com/CJ-1998/Image-Module>

주요 기능 및 담당 역할

• 프록시 캐싱 서버 구현

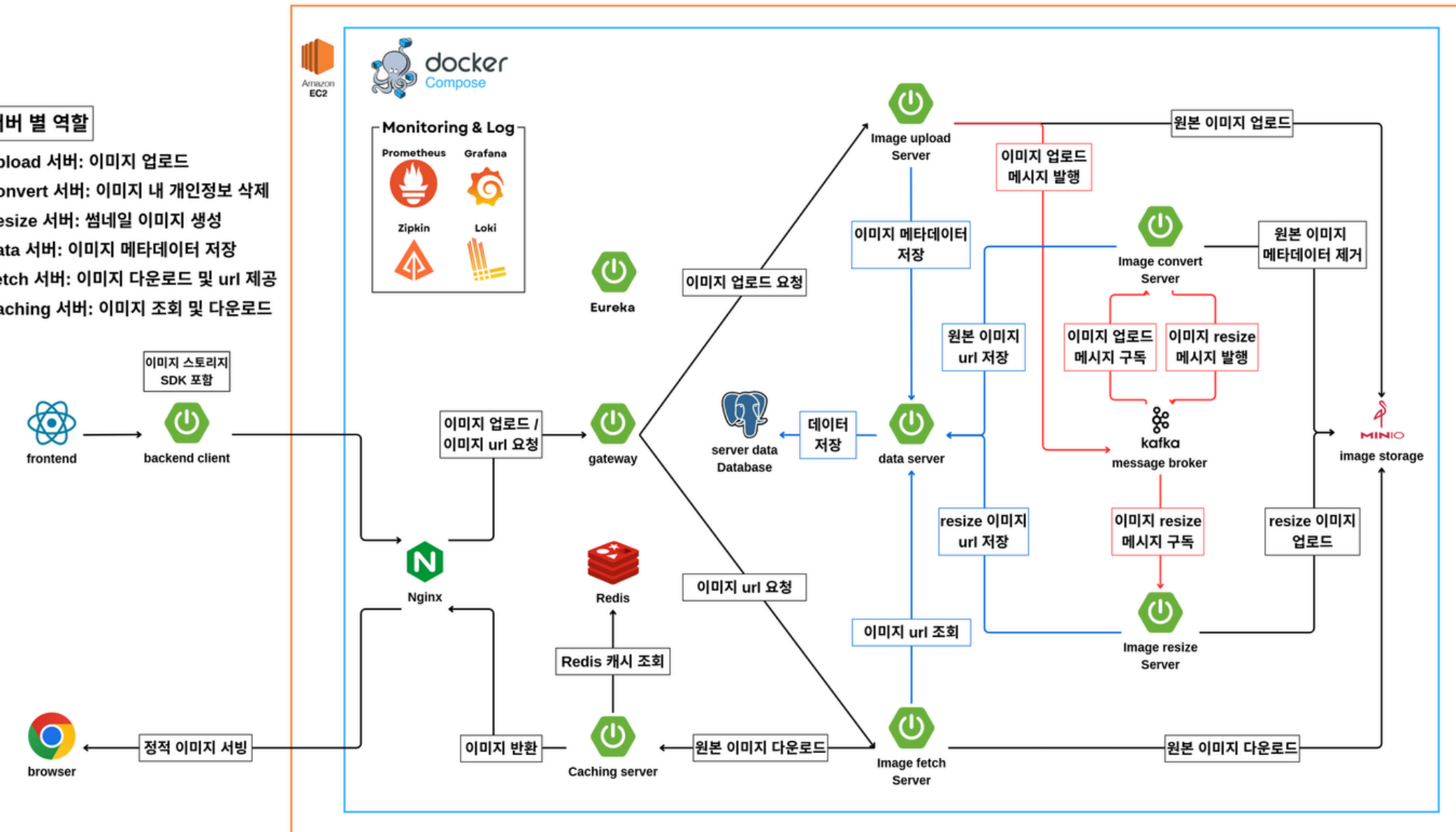
- 이미지 조회 및 다운로드 기능 구현
- Redis를 활용한 이미지 경로 캐싱 기능 구현
- 스트리밍 리팩토링으로 메모리 사용량 최적화
- Nginx 리버스 프록시 구축 통한 WAS 스레드 부하 분산

• Backend Client 라이브러리 구현

- Github & jitpack을 활용한 이미지 업로드 및 CDN URL 조회 라이브러리 구현

서버 별 역할

Upload 서버: 이미지 업로드
Convert 서버: 이미지 내 개인정보 삭제
Resize 서버: 썸네일 이미지 생성
Data 서버: 이미지 메타데이터 저장
Fetch 서버: 이미지 다운로드 및 url 제공
Caching 서버: 이미지 조회 및 다운로드



4# Storage Service

백엔드 팀의 이미지 관련 처리 돕는 이미지 처리 모듈

1) Redis 기반 캐싱 및 분산 락을 통한 동시성 제어

• 상황

- 다중 이미지 조회 시 매번 원본 저장소 및 DB에 접근하게 됨
- 동일 이미지에 대한 트래픽이 몰릴 경우 원본 서버에 부하가 집중되고 전체적인 응답 지연이 우려되는 상황
- 캐시 도입 후 캐시 비어있는 최초 시점에 동일 이미지 다수 요청 시 **캐시 스탬피드 현상 식별**

• 해결

- **Redis 활용한 이미지 위치 캐싱**을 도입하여 DB 조회 횟수를 줄임
- 다수 스레드의 동시 접근 시 중복 I/O를 방지하기 위해 **Redis Lock을 이용한 동시성 제어** 로직을 아키텍처에 구현

• 결과

- 불필요한 중복 I/O를 원천 차단하여 데이터 정합성을 보장
- 이미지 처리량을 기존 **71.4/s에서 264.1/s로 약 3.7배 향상**

동일 이미지 스레드 100개가 10번
반복 부하 테스트 시 결과

Redis Lock 구현 전 처리량 : 71.4/s

Max	Std. Dev.	Error %	Throughput
7938	1813.69	0.00%	71.4/sec
7938	1813.69	0.00%	71.4/sec

Redis Lock 구현 후 처리량 : 264.1/s

Max	Std. Dev.	Error %	Throughput
869	138.94	0.00%	264.1/sec
869	138.94	0.00%	264.1/sec

4# Storage Service

백엔드 팀의 이미지 관련 처리 돕는 이미지 처리 모듈

2) 스트리밍 리팩토링을 통한 JVM 메모리 사용량 개선

• 상황

- 파일을 **byte[] 배열**로 한 번에 메모리에 로드하는 기존 방식
- 다중 파일 요청 시 **JVM 힙 메모리가 급증**하고 빈번한 Garbage Collection이 발생하여 응답 지연 및 **Out of Memory 위험** 존재

• 해결

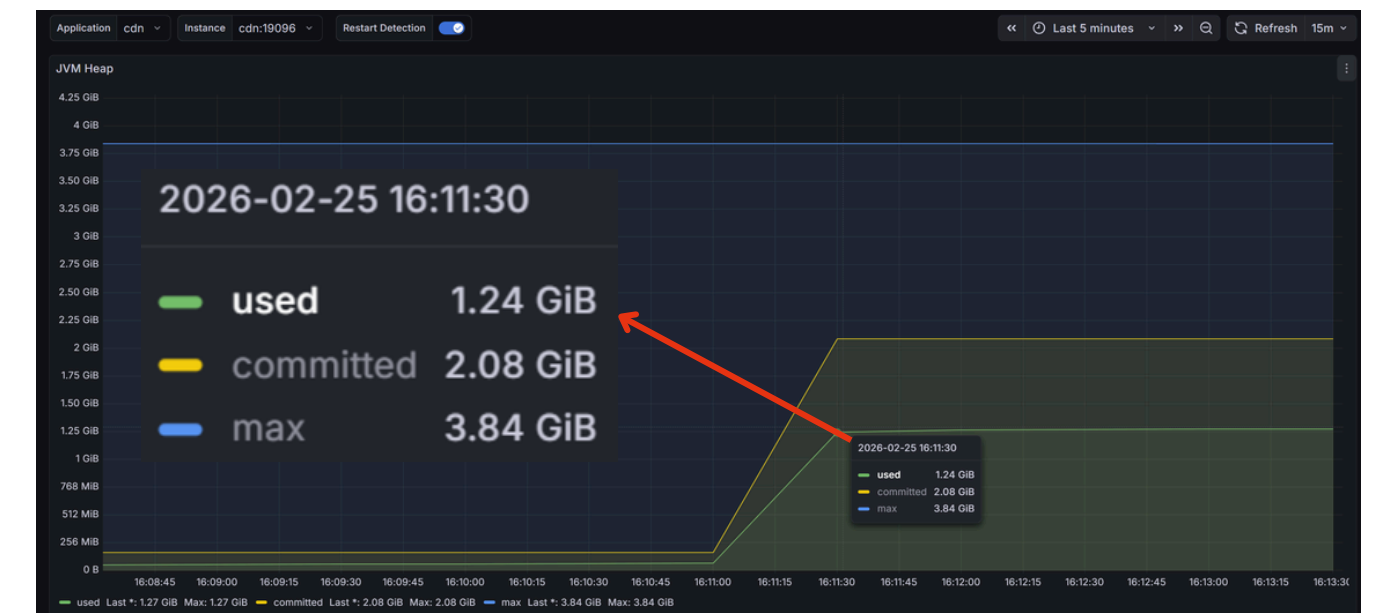
- **Resource** 인터페이스 활용
- 버퍼 단위의 데이터를 네트워크 소켓으로 직접 전달하는 **스트리밍 방식**으로 I/O 로직을 리팩토링

• 결과

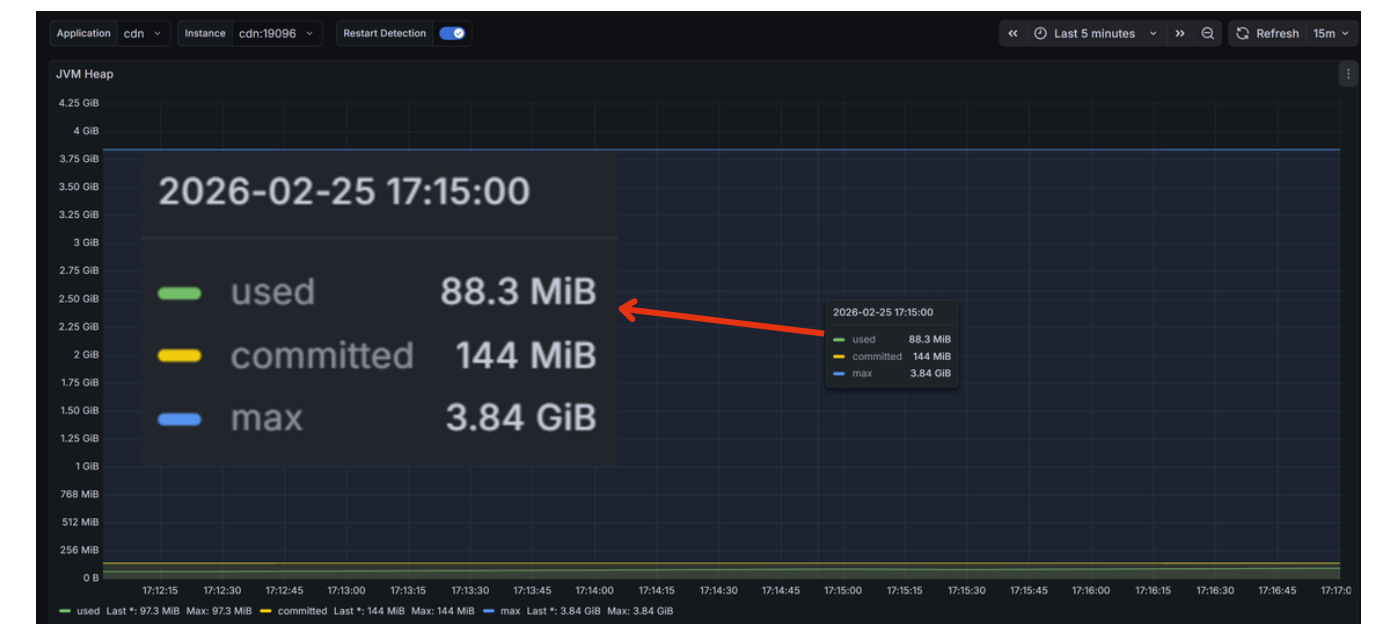
- 파일 크기와 무관하게 **메모리 점유를 일정하게 유지**
- 메모리 사용량을 **1.24GB에서 88.3MB로 약 93% 대폭 감소**시킴

5.97MB 이미지 스레드 100개
동시 요청 부하 테스트 시 결과

byte[] 방식 JVM Heap 메모리 사용량 : 1.24GB



Resource 방식 JVM Heap 메모리 사용량 : 88.3MB



4# Storage Service

백엔드 팀의 이미지 관련 처리 돕는 이미지 처리 모듈

3) 인프라 역할 분리를 통한 스레드 사용량 최적화

• 상황

- 정적 파일 서빙과 동적 비즈니스 로직 처리가 혼재됨
- WAS가 이미지 파일 I/O까지 처리하느라 비즈니스 로직 처리용 **스레드 부족 현상 발생**

• 해결

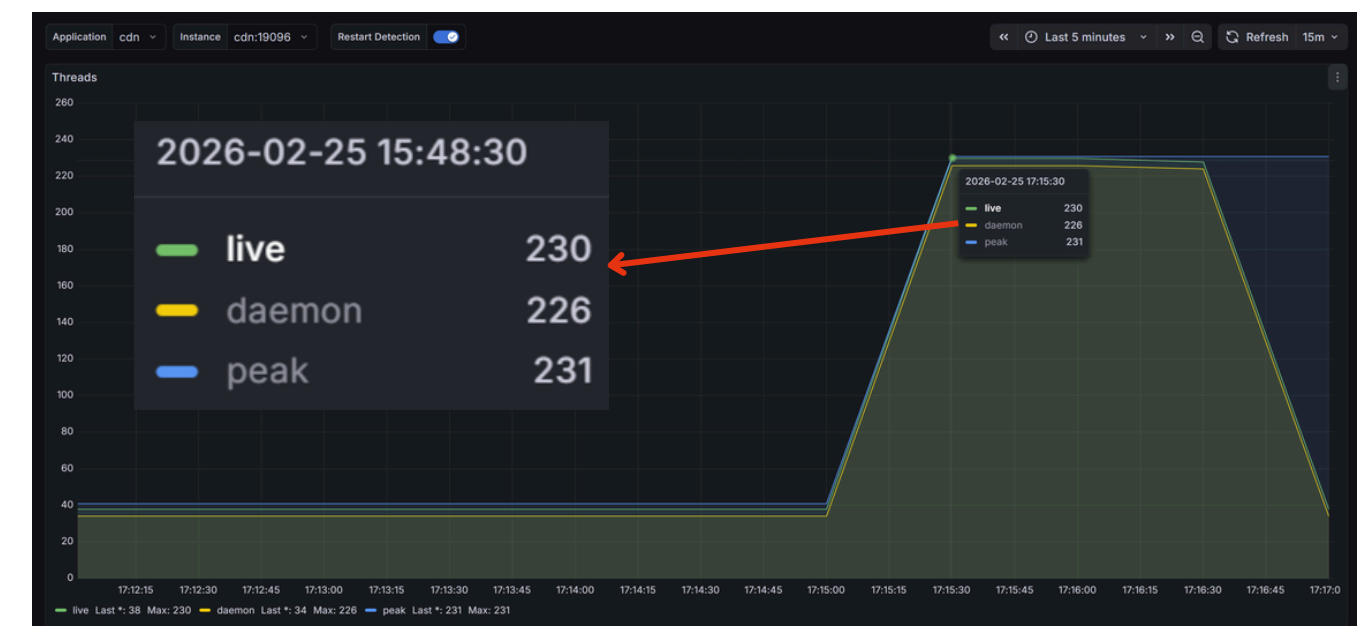
- 서버 앞단에 **Nginx 리버스 프록시**를 구축하여 정적 파일 Nginx 캐싱을 적용
- **Sendfile 최적화**를 통해 커널 영역에서 소켓으로 데이터를 직접 전달하도록 인프라 설계

• 결과

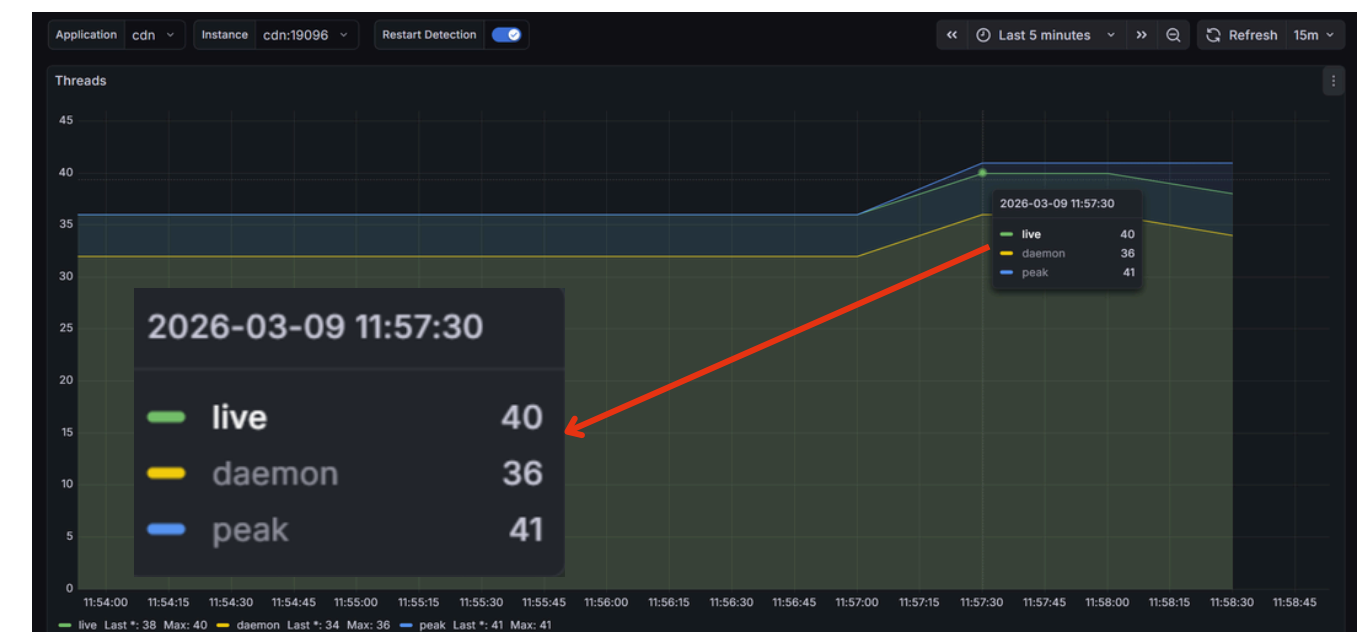
- 단순 이미지 요청이 WAS에 도달하는 것을 차단
- WAS 스레드 사용량을 **230개에서 40개로 약 82% 감소**
- 처리량을 **23.4/s에서 35.1/s로 50% 향상**해 안정성을 확보

5.97MB 이미지 스레드 100개
동시 요청 부하 테스트 시 결과

이전 로직 Threads 사용량 : 230개



Nginx 적용 후 Threads 사용량 : 40



PROJECTS

AskPlace

실시간 여행지 영상 확인 가능한 여행 계획 플랫폼

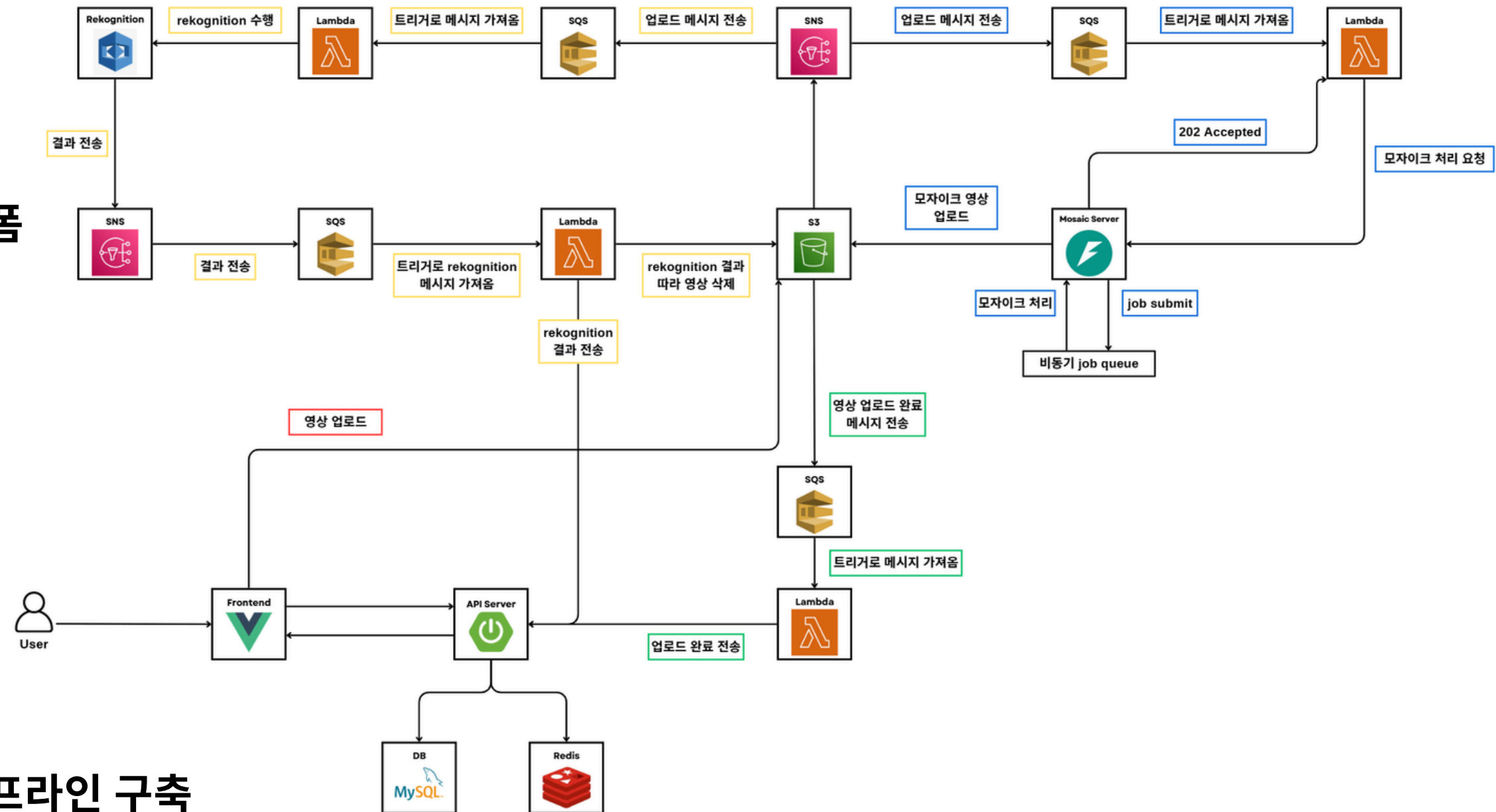
기간 : 25.11 ~ 25.12 (6주))

팀 구성 : Full Stack 2명

Github : <https://github.com/AskPlace>

주요 기능 및 담당 역할

- **AWS 활용한 이벤트 기반 미디어 처리 파이프라인 구축**
 - AWS S3 업로드 이벤트를 트리거로 SNS, SQS, Lambda 연동한 이벤트 기반 비동기 분산 아키텍처 설계 및 구축
- **AI 모자이크 파이썬 서버 구현**
 - FastAPI를 활용한 파이썬 웹 서버 구현
 - Yolo11n 모델 + KCF tracker를 통한 사람 얼굴 트래킹 후 모자이크 로직 구현



AskPlace

실시간 여행지 영상 확인 가능한 여행 계획 플랫폼

1) AWS 이벤트 기반 아키텍처로 미디어 처리 병목 해결

- 상황
 - 기존 영상 업로드 시 **AI 서버가 모자이크 처리**를 수행하는 구조
 - **유해 영상 판별** 요구사항 추가됨
 - 두 가지 무거운 미디어 작업 순차적으로 실행 시 **메인 서버의 응답 속도 저하** 예상
- 해결
 - AWS S3 업로드 이벤트를 트리거로 하여 **SNS, SQS, Lambda**를 연동한 **이벤트 기반 아키텍처 설계**
 - S3 이벤트를 SNS가 수신하면 한쪽은 **AWS Rekognition**으로 **유해성을 판별**
 - 다른 한쪽은 **SQS**를 거쳐 **AI 이미지 처리 서버**로 작업을 할당하도록 비동기화
- 결과
 - **무거운 영상 처리 작업을 메인 서버에서 분리**
 - 핵심 비즈니스 로직의 빠른 응답성과 안정성을 확보

AWS Rekognition 유해 영상 판별

▶ 타임스탬프	메시지
▶ 2025-12-25T22:28:13.109Z	INIT_START Runtime Version: python:3.14.v32 Runtime Version ARN: arn:aws:lambda:ap
▶ 2025-12-25T22:28:13.647Z	START RequestId: 2b35e329-a7e2-536d-938e-82ee1b5a3bfe Version: \$LATEST
▶ 2025-12-25T22:28:13.647Z	[*] 결과 조회 중: 73df66693c15c20fa8cbd3e3830572c4d1ff0fe6a431e7fc6d796133ccf4ee72
▶ 2025-12-25T22:28:13.972Z	⚠ 유해 콘텐츠 감지됨! 사유: Drugs & Tobacco (78.4%)
▶ 2025-12-25T22:28:14.234Z	📄 파일 삭제 완료: videos/fa009f0b-355d-4e93-9811-cc87619a9e48_독전영상.mp4

SQS 이미지 업로드 메시지 수신

메시지 (3)					세부 정보 보기	삭제
🔍 메시지 검색					< 1 >	⚙
☐	ID	전송됨	크기	수신 수		
☐	8092acde-ac2f-489d-9054-af2c9fd1735f	2025-12-21T12:28+09:00	226바이트	2		
☐	a243ddd3-6cbf-4965-9e69-0d4b783f509b	2025-12-21T12:32+09:00	817바이트	2		
☐	6b070bba-39e5-43c9-b2ff-09180f8acde4	2025-12-21T12:34+09:00	817바이트	1		

AskPlace

실시간 여행지 영상 확인 가능한 여행 계획 플랫폼

2) Bulk 연산 및 비동기 분리로 API 응답 속도 최적화

• 상황

- 여행 계획 상세 조회 및 저장 시, **잡은 외부 API 호출**과 **개별적인 DB 업데이트** 작업 수행
- I/O 바운드 작업이 메인 스레드에서 동기적으로 실행되어 심각한 응답 지연이 발생

• 해결

- DB 접근 시 단건 처리 방식이 아닌 **Bulk Insert 및 Update**를 적용하여 I/O 발생 횟수를 최소화
- **외부 API 호출 로직을 별도의 비동기 작업으로 분리**하여 메인 스레드의 블로킹 현상을 방지

• 결과

- 여행 계획 조회 API의 **응답 속도를 2.23배 개선**
- 여행 계획 저장 API **응답 속도를 1.12배 개선**

외부 API 동기 처리 및 DB update 동기 처리
여행 계획 조회 속도 : 2700ms

```
: Initializing Spring DispatcherServlet 'dispatcherServlet'
: Initializing Servlet 'dispatcherServlet'
: Completed initialization in 2 ms
: HikariPool-1 - Starting...
: HikariPool-1 - Added connection com.mysql.cj.jdbc.Connection
: HikariPool-1 - Start completed.
: 하루 계획 상세 정보 가져오는 시간: 2700 ms
```

외부 API 비동기 처리 및 DB update Bulk 처리
여행 계획 조회 속도 : 1210ms

```
: Initializing Spring DispatcherServlet 'dispatcherServlet'
: Initializing Servlet 'dispatcherServlet'
: Completed initialization in 3 ms
: HikariPool-1 - Starting...
: HikariPool-1 - Added connection com.mysql.cj.jdbc.Connection
: HikariPool-1 - Start completed.
: 하루 계획 상세 정보 가져오는 시간: 1210 ms
: [Async] Success bulk update place description
```

감사합니다